YENEPOYA

(DEEMED TO BE UNIVERSITY)

Project – High Level Design

on

AI-BASED EDGE OBJECT DETECTION FOR HEALTHCARE SYSTEM

Student Name: MUHAMMED.CH

Industry Mentor:

Guided by:

Table of Contents

1. Introduction

The integration of Artificial Intelligence with edge computing has enabled intelligent systems to operate directly on local devices without relying on cloud infrastructure. This paradigm, known as **Edge AI**, ensures faster processing, reduced latency, and enhanced data privacy—critical factors in healthcare environments.

The **AI-Based Edge Object Detection for Healthcare System** is designed to perform real-time image classification using a lightweight **MobileNet TensorFlow Lite (TFLite)** model. The system captures live video input through a webcam, processes the frames locally, and predicts objects without requiring internet connectivity.

Additionally, the system records detected objects along with timestamps, enabling monitoring and analysis of healthcare-related activities.

1.1 Scope of the Document

This document provides a comprehensive High-Level Design of the system, covering:

- System architecture
- Application workflow
- Information and process flow
- Component-level design
- Data handling mechanisms
- Interfaces and performance considerations

This document focuses on **overall system structure**, not detailed code implementation.

1.2 System Overview

The system performs **real-time object classification** using a pre-trained lightweight deep learning model.

Key Functionalities

- Capture real-time video input
- Perform image preprocessing
- Run inference using TFLite model
- Display detected object labels

Technologies Used

- Python
- TensorFlow Lite (TFLite)
- OpenCV
- NumPy

1.3 Intended Audience

- Students and Developers
- Project Evaluators and Examiners
- Technical Reviewers
- Academic Institutions

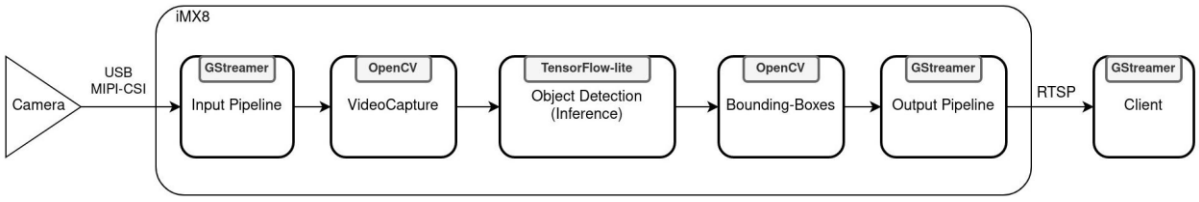
2. System Design

2.1 Application Design

The system follows a modular layered architecture:

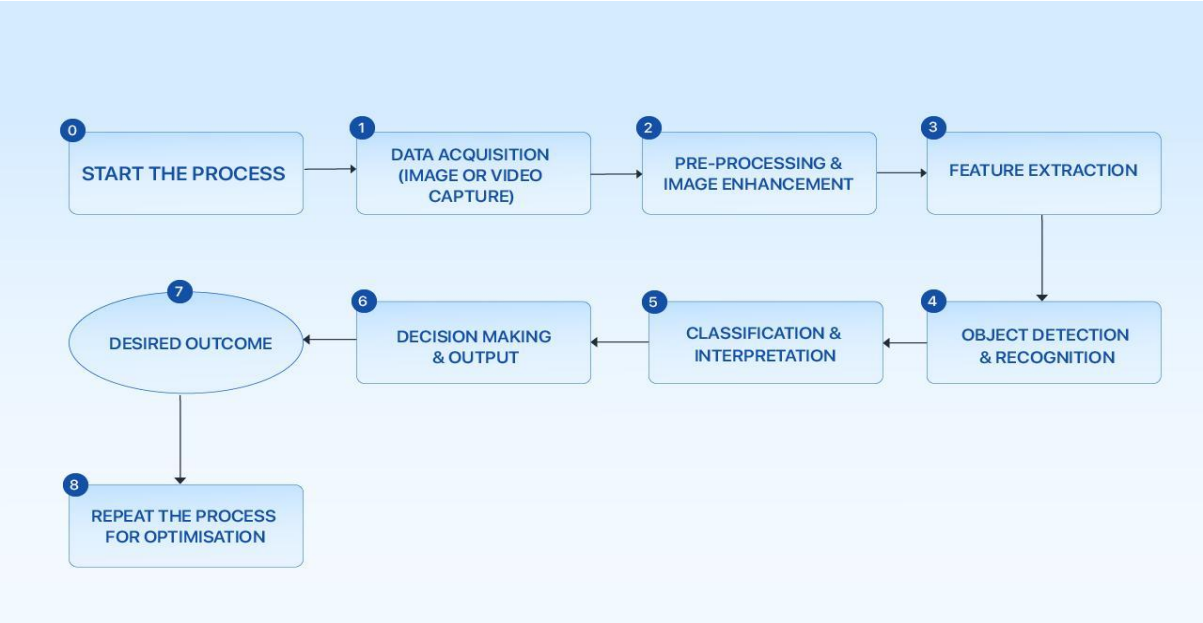
- **Input Layer** → Webcam input
- **Processing Layer** → Image preprocessing
- **Inference Layer** → TFLite MobileNet model
- **Output Layer** → Display and storage

2.2 Process Flow



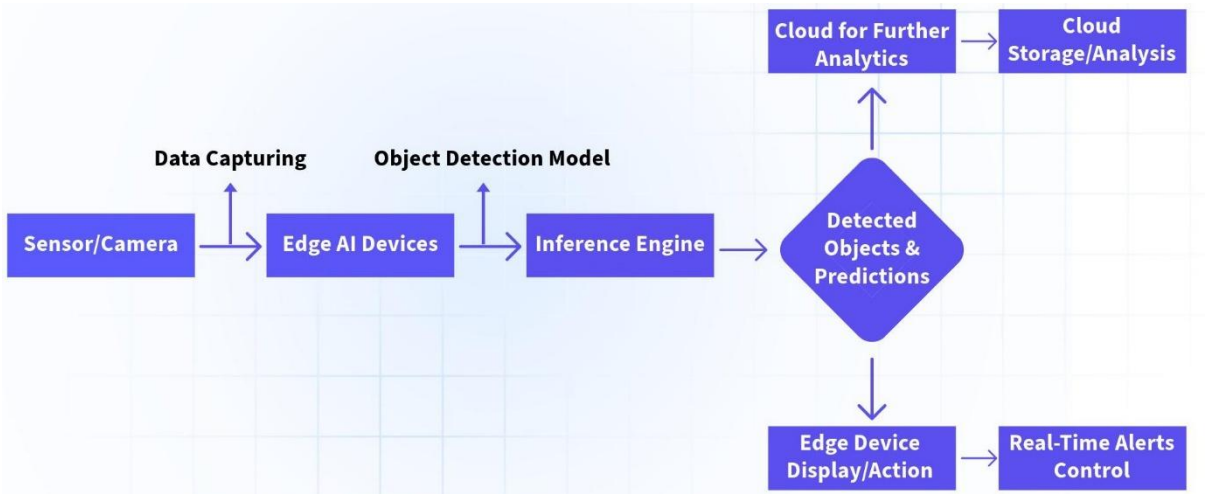
- Capture image frame from webcam
- Resize and normalize the image
- Feed processed image into TFLite model
- Perform inference (prediction)
- Extract class label and confidence score
- Display result on screen
- Store detection data in CSV file

2.3Information Flow



- User starts the application
- Webcam captures continuous frames
- Frames are passed to preprocessing module
- Preprocessed data is fed into the inference engine
- Model generates predictions
- Output is displayed and stored locally

2.4 Components Design



- **Camera Module**
  Captures real-time video input using webcam
- **Preprocessing Module**
  Resizes, normalizes, and prepares image data
- **Inference Engine (TFLite Model)**
  Executes MobileNet model for prediction
- **Prediction Module**
  Extracts class label and confidence score
- **Display Module**
  Shows results on screen using OpenCV
- **Storage Module**
  Saves detection data (timestamp, label, confidence)

2.5 Key Design Considerations

- Lightweight model for edge deployment
- Real-time performance optimization
- Data privacy (no cloud dependency)
- Low computational resource usage
- Scalability for future enhancements

2.6 API Catalogue

API Name	Description	Input	Output
capture_frame()	Captures image frame	Webcam	Image
preprocess()	Processes image	Raw image	Processed image
run_inference ()	Executes model	Image	Prediction
display_output ()	Displays result	Label	Screen output
save_data ()	Stores detection	Label, Time	CSV entry

3. Data Design

3.1 Data Model

Field Name	Type	Description
timestamp	String	Time of detection
label	String	Detected object
confidence	Float	Prediction score

3.2 Data Access Mechanism

- Data is stored in CSV format
- Accessed using Python file handling
- Append operation for real-time logging

3.3 Data Retention Policies

- Data stored locally on device
- No sensitive personal data involved
- Can be modified or deleted anytime

3.4 Data Migration

- Manual transfer of CSV files
- Future integration with databases (SQL/NoSQL)

4. Interfaces

- **User** **Interface**
  OpenCV display window
- **System** **Interface**
  Interaction between Python modules
- **External** **Interface**
  None (fully offline system)

5. State and Session Management

- Stateless architecture
- No user session required
- Each frame processed independently

6. Caching

- No caching implemented
- Future: caching prediction results

7. Non-Functional Requirements

7.1 Security Aspects

- Fully local processing
- No external data transmission
- Ensures data privacy

7.2 Performance Aspects

- Fast inference using TFLite
- Optimized CPU execution
- Real-time processing capability

8. References

- TensorFlow Lite Documentation
- OpenCV Documentation
- Python Official Documentation
- Edge AI Research Papers